



PROGRAMAÇÃO ORIENTADA À OBJETOS

AULA 04



AGENDA

01

MÉTODOS EM JAVA

02

EXERCÍCIOS



01

Métodos em Java

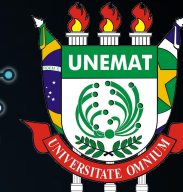




- Métodos são trechos de código que permitem modularizar um sistema, isto é, são pequenos blocos que, juntos, compõem um sistema maior;
- Os métodos recebem um determinado nome e podem ser chamados várias vezes durante a execução de uma classe, ou seja, é uma sub-rotina que pode ser invocada toda vez que sua funcionalidade for necessária;
- Os principais motivos que levam a utilizar métodos se referem à redução do tamanho total de código, à melhoria da modularização do sistema e à facilitação e agilização do processo de manutenção;



- Imagine um grande sistema em que existe a necessidade de verificar se uma data é válida. Imagine ainda que em diversas aplicações isso deve ser realizado, isto é, a mesma validação de data deve ser feita em diversas classes;
- Um programador pouco experiente talvez repita o mesmo trecho de programa em várias classes no sistema. Isso não é bom, pois, além de o código dos programas ficar maior, pode ser preciso alterar algo;
- Por exemplo, a data era considerada com dois dígitos para o ano e depois foi necessário tratá-la para ser aceita com quatro dígitos. É preciso alterar todas as classes que utilizam essa validação;



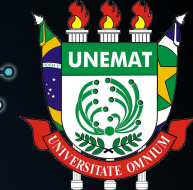
- Suponha que em 15 classes diferentes, com diversos métodos distintos em cada uma, essa rotina fosse utilizada. Seria necessário alterar e compilar as 15 classes novamente;
- Por outro lado, se essa rotina de validação estivesse armazenada na forma de método em uma classe, todas as outras classes poderiam usar (importar) esse método;
- E melhor ainda, caso alterações no método fossem necessárias, apenas ele seria alterado e todas as classes que o utilizam seriam atualizadas automaticamente;

01 Métodos em Java



- Um método pode invocar outro método, isto é, durante a execução do **método1** pode ser necessária a execução do **método2** que pode invocar o **método3** e assim por diante. Todo método possui uma declaração e um corpo, cuja estrutura simplificada é formada por:

```
qualificador tipo-do-retorno nome-do-metodo ([lista-de-  
parametros]){  
    <codigos-do-corpo>  
}
```



- **Qualificador:** define a visibilidade do método. A Oracle o define como “nível de acesso” do método. Trata-se de uma forma de especificar se o método é visível apenas à própria classe em que está declarado ou pode ser visualizado por classes externas. Pode ser do tipo:
 - ***public***: o método é visível por qualquer classe. É o qualificador mais aberto no sentido de que qualquer classe pode usar este método;
 - ***private***: o método é visível apenas pela própria classe. É o qualificador mais restritivo;
 - ***protected***: o método é visível pela própria classe, por suas subclasses e pelas classes do mesmo pacote.



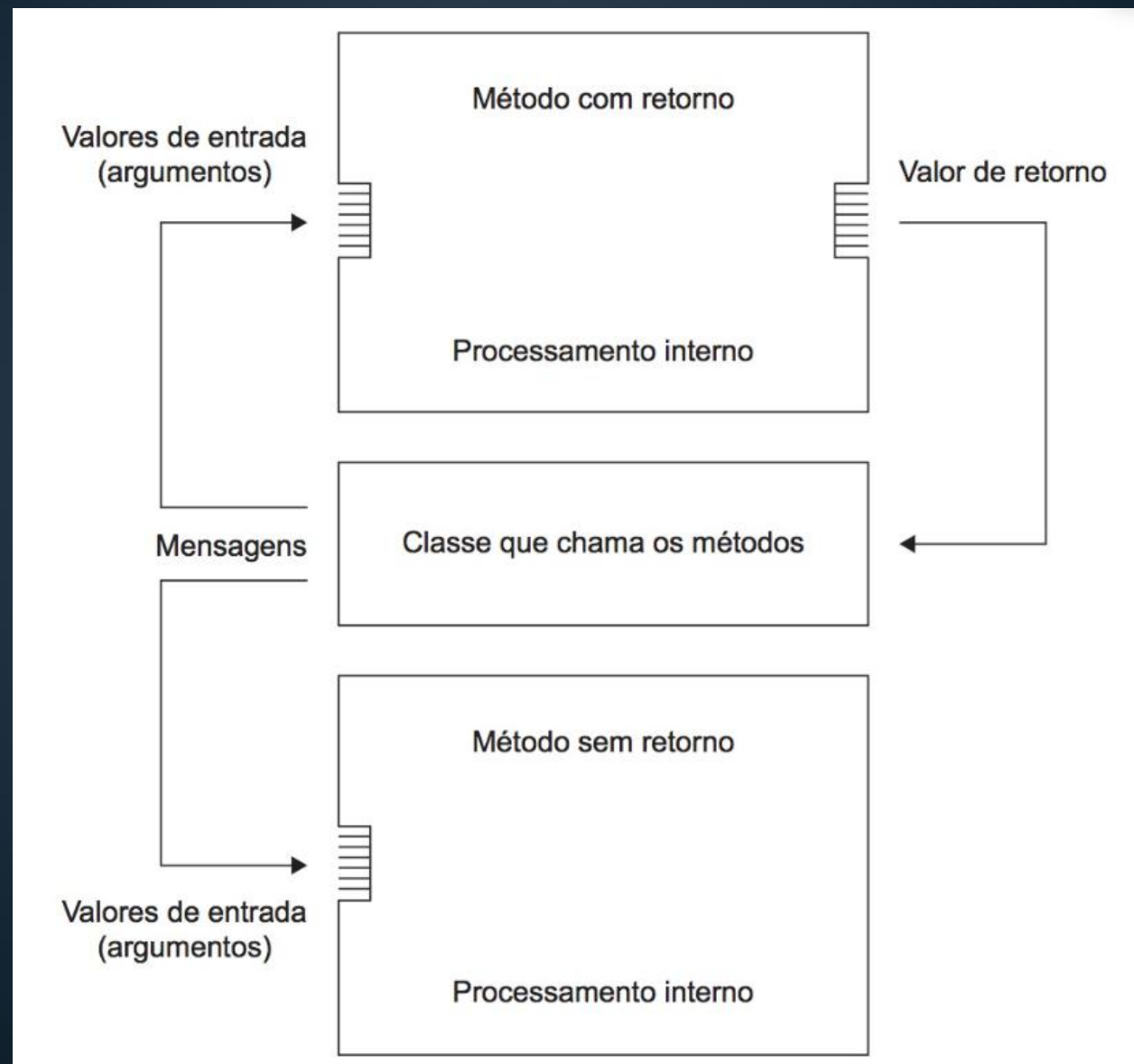
- **Tipo-de-retorno:** refere-se ao tipo de dado retornado pelo método. Métodos que não retornam valores devem possuir nesse parâmetro a palavra *void*;
- Sempre que *void* for usada em uma declaração de método. nenhum valor é retornado após sua execução, isto é, o método atua como uma *procedure* de outras linguagens de programação;
- Um método pode ter como retorno qualquer tipo primitivo (*int*, *float*, *double* etc), um *array* ou ainda um objeto qualquer;



- **Nome-do-método:** pode ser qualquer palavra ou frase, desde que iniciada por uma letra. Se o nome for uma frase, não podem existir espaços em branco entre as palavras;
- Como padrão da linguagem Java, o nome de um método sempre inicia com uma palavra com letras minúsculas. Se outras palavras forem necessárias, elas devem iniciar com maiúsculas;
- É importante dar aos métodos nomes sugestivos, ou seja, que identificam facilmente a tarefa executada por eles;

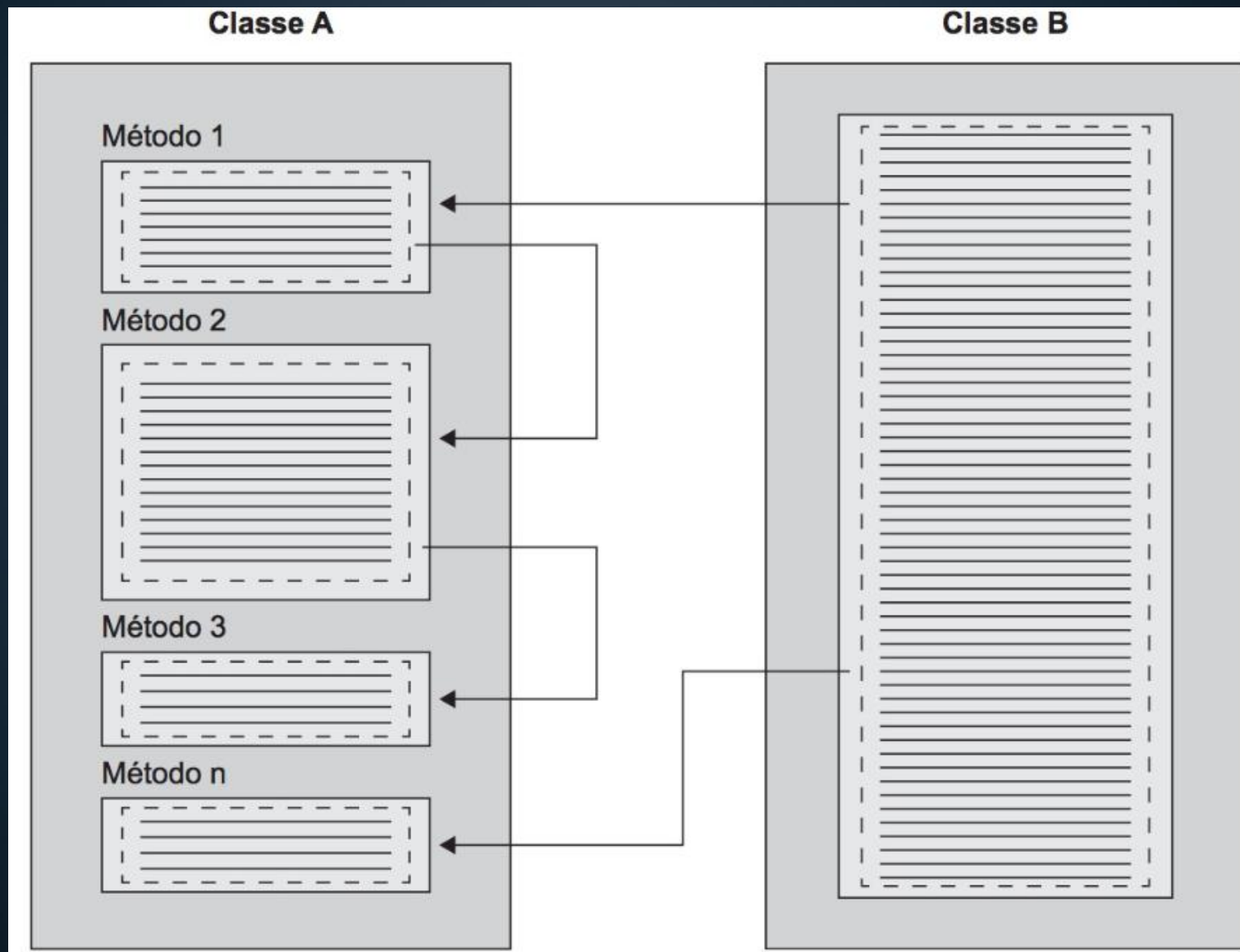


- **Lista-de-parâmetros:** trata-se de uma lista de variáveis opcionais, que podem ser recebidas pelo método para tratamento interno;
- Quando um método é invocado (chamado), ele pode receber valores de quem o chamou. Esses valores podem ser manipulados internamente e devolvidos ao emissor da solicitação;
- Esse processo pode ser comparado ao de uma fabricação industrial: entra matéria-prima (os valores passados ao método) e sai um produto acabado (o retorno do método);





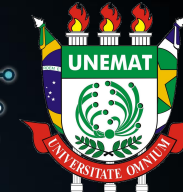
- **Códigos do corpo:** trata-se dos códigos implementados em Java que realizam os processos internos e retornam os valores desejados, isto é, constituem o programa do método;
- O nome do método e a lista de parâmetros formam a **assinatura** do método, algo que o identifica de maneira única. Toda vez que um método for invocado (chamado), a assinatura deve ser obedecida, uma maneira que torna possível identificar um método em relação aos demais;
- Se a assinatura for diferente, ou seja, se qualquer um dos componentes não coincidir com a declaração, o método não pode ser utilizado;





- Não retornam valores e são semelhantes às *procedures* encontradas em outras linguagens de programação, como Pascal. Os métodos que não retornam valores devem ser definidos como **void**. Dessa forma, todos os métodos sem retorno acompanham a seguinte estrutura:

```
qualificador void nome-do-metodo ([lista-de-parametros]){  
    //códigos do corpo  
}
```

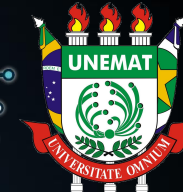


```
1 //Classe com uso de Método sem Retorno
2 class MetodoSemRetorno {
3     public static void main (String args[]) {
4         imprimir();
5         imprimirTexto("Ola");
6         somar(10,20);
7         somar(10,20,30);
8     }
9
10    public static void imprimir() {
11        System.out.println("Aprendendo métodos em Java");
12    }
13
14    public static void imprimirTexto(String texto) {
15        System.out.println(texto);
16    }
17
18    public static void somar(int a, int b) {
19        System.out.println("Soma de 2 números: " + (a + b));
20    }
21
22    public static void somar(int a, int b, int c) {
23        System.out.println("Soma de 3 números: " + (a + b + c));
24    }
25 }
```



- Métodos com retorno seguem a seguinte estrutura:

```
qualificador tipo-do-retorno nome-do-metodo ([lista-de-  
parametros]){  
    //códigos do corpo  
    return (valor-de-retorno)  
}
```



```
1 //Classe com uso de Método com Retorno
2 class MetodoComRetorno {
3     public static void main (String args[]) {
4         System.out.println(MetodoComRetorno.somar(2,3));
5         System.out.println(MetodoComRetorno.mostrarPares(100));
6         System.out.println(MetodoComRetorno.contarLetrasA("Banana"));
7     }
8
9     public static double somar(int a, int b) {
10         return a + b;
11     }
12
13     public static String mostrarPares(int valor) {
14         String retorno = "";
15         for (int a = 0; a <= valor; a = a+2) {
16             retorno += a + " ";
17         }
18         return retorno;
19     }
20
21     public static int contarLetrasA(String palavra) {
22         int qtde = 0;
23         palavra = palavra.toUpperCase();
24         for (int a = 0; a < palavra.length(); a++) {
25             if (palavra.charAt(a) == 'A') {
26                 qtde++;
27             }
28         }
29         return qtde;
30     }
31 }
```



- Os programas são geralmente estruturados com métodos que chamam uns aos outros, o que facilita a resolução de muitos problemas, além de reduzir consideravelmente o tamanho do código;
- A recursividade ocorre quando um método chama a si próprio, direta ou indiretamente, por meio de outro método;
- Para entender corretamente o funcionamento desse tipo de método, é necessária muita dedicação por parte dos desenvolvedores, pois sua utilização pode ser extremamente complexa;



```
1 //Classe com uso de Método com Recursividade
2 class MetodoComRecursividade {
3     public static void main (String args[]) {
4         for (int i = 0; i <= 10; i++) {
5             System.out.println(i + "! = " + calcularFatorial(i));
6         }
7     }
8
9     public static long calcularFatorial(long num) {
10        if (num <= 1) {
11            return (1);
12        } else {
13            return (num * calcularFatorial(num - 1));
14        }
15    }
16 }
```



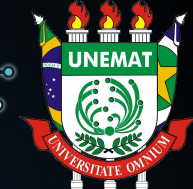
- A linguagem Java permite que vários métodos sejam definidos com o mesmo nome, desde que eles tenham uma assinatura diferente, ou seja, essas diferenças podem ser com base no número, nos tipos ou na ordem de parâmetros recebidos;
- Quando um método sobrecarregado é chamado, o compilador avalia e seleciona o método mais adequado à situação, examinando a assinatura correspondente, portanto os métodos sobrecarregados são utilizados para a realização de tarefas semelhantes sobre tipos de dados diferentes;



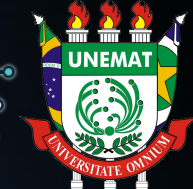
- Sem nos darmos conta disso, já utilizamos a sobrecarga várias vezes. Considere **System.out.println()**. O método **println** pode receber diversos dados como parâmetro, isto é, ora pode-se enviar um dado **inteiro** para ser impresso, ora pode ser enviado um tipo **double**, ou ainda **String**;
- Isso foge às regras de definição dos métodos, uma vez que ele deve receber o mesmo tipo de dado declarado. Sendo assim, como o método **println()** pode receber dados de tipos diferentes? Isso é possível por meio da sobrecarga de métodos;



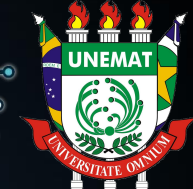
```
1 //Classe com uso de Método sem Retorno
2 class MetodoSemRetorno {
3     public static void main (String args[]) {
4         imprimir();
5         imprimirTexto("Ola");
6         somar(10,20);
7         somar(10,20,30);
8     }
9
10    public static void imprimir() {
11        System.out.println("Aprendendo métodos em Java");
12    }
13
14    public static void imprimirTexto(String texto) {
15        System.out.println(texto);
16    }
17
18    public static void somar(int a, int b) {
19        System.out.println("Soma de 2 números: " + (a + b));
20    }
21
22    public static void somar(int a, int b, int c) {
23        System.out.println("Soma de 3 números: " + (a + b + c));
24    }
25 }
```



- Até o momento trabalhamos com acesso a métodos da própria classe, com exceção dos métodos internos da própria linguagem Java. Essa característica, apesar de útil, não é exatamente a mais usada em Java;
- O mais comum é que um método seja acessado externamente, isto é, ele é declarado em uma classe externa e sua funcionalidade é reaproveitada por qualquer outra classe;
- Já utilizamos métodos externos diversas vezes, como, por exemplo, os das classes *Math* e *String*;



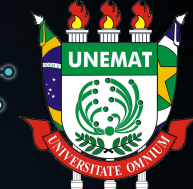
- Qualquer método do tipo static criado em uma classe pode ser chamado a partir de outra classe usando o formato:
 - **nome-da-classe.nome-do-método;**
- Com isso um método necessita ser criado apenas uma vez em uma única classe. Esse recurso é muito importante, pois, uma vez que o método foi criado, ele pode ser reutilizado em outra aplicação; basta chamar a classe em que ele se encontra;



```
1 //Classe com uso de Método de outras Classes
2 class AcessoMetodosOutrasClasses {
3     public static void main (String args[]) {
4         MetodoSemRetorno.imprimirTexto("Usando métodos de outra classe.");
5         MetodoSemRetorno.somar(20,50);
6         System.out.println(MetodoComRetorno.contarLetrasA("Macacada"));
7         System.out.println(MetodoComRetorno.mostrarPares(50));
8     }
9 }
```



```
1 //Classe com uso de Método com Arrays
2 import java.util.Arrays;
3 class MetodoComArray {
4     /**
5      * Recebe um array de números e retorna o maior deles, deve
6      * conter pelo menos um elemento no array senão gera erro.
7      *
8      * @param numeros = os números a serem comparados
9      * @return = o maior número
10     */
11     public static double buscarMaior (double[] numeros) {
12         double maior = numeros[0];
13         for (int i = 1; i < numeros.length; i++){
14             maior = Math.max(maior, numeros[i]);
15         }
16         return maior;
17     }
18
19     /**
20      * Recebe um array de palavras e retorna ordenados
21      * em ordem crescente.
22      *
23      * @param palavras = as palavras a serem ordenadas
24      * @return = o array ordenado
25     */
26     public static String[] ordenarPalavras (String[] palavras) {
27         Arrays.sort(palavras);
28         return palavras;
29     }
30 }
```

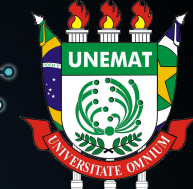


```
1 //Classe com uso de Método de outras Classes
2 class AcessoMetodoComArray {
3     public static void main (String args[]) {
4         double[] numeros = {12.4, 3.4, 9, 7.3, 6.6};
5         double maior = MetodoComArray.buscarMaior(numeros);
6         System.out.println(maior);
7
8         String[] nomes = {"Ivone", "Carlos", "Lila", "Alzira", "Fabio", "Rita"};
9         String[] retorno = MetodoComArray.ordenarPalavras(nomes);
10        for (String nome : retorno){
11            System.out.println(nome);
12        }
13    }
14 }
```



```
1 //Classe com uso de Método com Arrays
2 import java.util.Arrays;
3 class MetodoComArrayPrivate {
4     /**
5      * Acessa método privado
6      *
7      * @param numeros = os números a serem comparados
8      * @return = o maior número
9      */
10    public static double getBuscarMaior (double[] numeros) {
11        return buscarMaior(numeros);
12    }
13
14    /**
15     * Acessa método privado
16     *
17     * @param palavras = as palavras a serem ordenadas
18     * @return = o array ordenado
19     */
20    public static String[] getOrdenarPalavras (String[] palavras) {
21        return ordenarPalavras(palavras);
22    }
23 }
```

```
23
24 /**
25  * Recebe um array de números e retorna o maior deles, deve
26  * conter pelo menos um elemento no array senão gera erro.
27  *
28  * @param numeros = os números a serem comparados
29  * @return = o maior número
30  */
31 private static double buscarMaior (double[] numeros) {
32     double maior = numeros[0];
33     for (int i = 1; i < numeros.length; i++){
34         maior = Math.max(maior, numeros[i]);
35     }
36     return maior;
37 }
38
39 /**
40  * Recebe um array de palavras e retorna ordenados
41  * em ordem crescente.
42  *
43  * @param palavras = as palavras a serem ordenadas
44  * @return = o array ordenado
45  */
46 private static String[] ordenarPalavras (String[] palavras) {
47     Arrays.sort(palavras);
48     return palavras;
49 }
50 }
```



```
1 //Classe com uso de Método de outras Classes
2 class AcessoMetodoComArrayPrivate {
3     public static void main (String args[]) {
4         double[] numeros = {12.4, 3.4, 9, 7.3, 6.6};
5         double maior = MetodoComArrayPrivate.getBuscarMaior(numeros);
6         System.out.println(maior);
7
8         String[] nomes = {"Ivone", "Carlos", "Lila", "Alzira", "Fabio", "Rita"};
9         String[] retorno = MetodoComArrayPrivate.getOrdenarPalavras(nomes);
10        for (String nome : retorno){
11            System.out.println(nome);
12        }
13    }
14 }
```



02

Exercícios





- Faça uma classe chamada Mes que possua um método chamado getMesPorExtenso que recebe um número inteiro, referente ao mês do ano, um código referente ao idioma (1 para português e 2 para inglês) e retorne o mês por extenso. A tabela a seguir ilustra alguns exemplos. Faça também outra classe para testar o método.

Número do Mês	Idioma	Retorno
1	1	Janeiro
2	2	February
3	1	Março
4	2	April
5	1	Maio
6	2	June

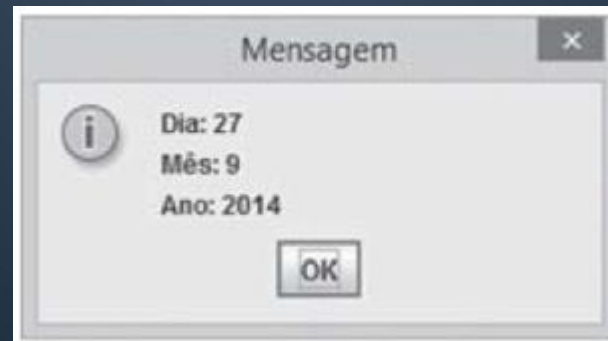


- Crie uma classe chamada Contagem que contenha quatro métodos com o nome “contar” utilizando o conceito da sobrecarga. A assinatura e função de cada método aparecem descritas na tabela a seguir. Faça também outra classe para testar o método.

Assinatura	Função
contar()	Apresenta em tela a contagem dos números de 1 a 10.
contar (int fim)	Apresenta em tela a contagem dos números de 1 até o valor definido pela variável fim.
contar (int inicio, int fim)	Apresenta em tela a contagem dos números com valor inicial definido pela variável inicio até o valor definido pela variável fim.
contar (int inicio, int fim, int pausa)	Apresenta em tela a contagem dos números com valor inicial definido pela variável inicio até o valor definido pela variável fim. O parâmetro pausa define a pausa (em segundos) que deve ser dada a cada valor impresso em tela.



- Crie uma classe chamada Data que contenha os métodos getDia, getMes e getAno. Esses métodos recebem uma data em formato texto no padrão “dd/mm/aaaa” e retornam um número inteiro correspondente ao dia, mês e ano existentes na data fornecida. Caso a data enviada esteja fora do padrão, os métodos, deve retornar o valor zero. Faça também outra classe para testar o método;





“A dúvida é o princípio da sabedoria.”

—Aristóteles